

TEST FOR ANYLOGISTIX API

The API info is here: <https://anylogistix.help/api/python/readme.html>

=====

PHASE 2 Flat AI Decision & Scenario Cloning

◀  ▶

Install the dependencies (All required for PHASE 2)

```
In [20]: # Install xlwings to interact natively with Microsoft Excel
!{sys.executable} -m pip install xlwings
```

Requirement already satisfied: xlwings in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (0.35.1)

Requirement already satisfied: pywin32>=224 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from xlwings) (311)

```
In [ ]: import os
import shutil
import xlwings as xw #For excel modifications
import time
from pprint import pprint

import re # Used for cleaning text strings
```

1. Define the Flat AI logic (3 predefined decisions)

```
In [26]: decision_options = [
    "DECISION 0: Increase Demand by 20% (Aggressive Growth)",
    "DECISION 1: Decrease Transport Costs by 15% (Cost Optimization)",
    "DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)"
]

print("--- FLAT AI: EVALUATING PREDEFINED DECISIONS ---")
for i, decision in enumerate(decision_options):
    print(f"[{i}] {decision}")

# For this 'flat AI' phase, we will simulate the AI choosing a decision automatically
# Let us assume the AI evaluates the current context and selects Decision 0.
selected_decision_index = 2

selected_decision_text = decision_options[selected_decision_index]

print(f"\nFLAT AI SELECTED: {selected_decision_text}")
```

--- FLAT AI: EVALUATING PREDEFINED DECISIONS ---

[0] DECISION 0: Increase Demand by 20% (Aggressive Growth)

[1] DECISION 1: Decrease Transport Costs by 15% (Cost Optimization)

[2] DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)

FLAT AI SELECTED: DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)

2. Flat AI Autonomous Data Modification

```
In [27]: # Define the paths for the original and the new modified scenario
ORIGINAL_EXCEL_FILENAME = "Global Network Examination.xlsx"
ORIGINAL_EXCEL_PATH = os.path.abspath(ORIGINAL_EXCEL_FILENAME)
MODIFIED_EXCEL_PATH = os.path.abspath(f"Modified_{ORIGINAL_EXCEL_FILENAME}")

print(f"Reading original scenario from: {ORIGINAL_EXCEL_PATH}")

# 1. Create a fresh copy of the original Excel to avoid corrupting the base one
try:
    shutil.copy2(ORIGINAL_EXCEL_PATH, MODIFIED_EXCEL_PATH)
    print("Created a clean copy of the scenario.")
except Exception as e:
    print(f"Error copying file: {e}")

print("Opening Microsoft Excel in the background to preserve strict formatting..")

# 2. Open Microsoft Excel silently (visible=False) so it doesn't interrupt you
app = xw.App(visible=False)

try:
    wb = app.books.open(MODIFIED_EXCEL_PATH)

    if selected_decision_index == 0:
        # DECISION 0: Increase Demand by 20%
        print(f"Applying {decision_options[selected_decision_index]}...")
        ws = wb.sheets['Demand']

        # Get the range of used columns and rows
        last_col = ws.used_range.last_cell.column
        last_row = ws.used_range.last_cell.row

        # Extract headers to find the right columns dynamically
        headers_raw = ws.range((1, 1), (1, last_col)).value
        headers = headers_raw if isinstance(headers_raw, list) else [headers_raw]

        if 'Col 7' in headers and 'Col 10' in headers:
            col7_idx = headers.index('Col 7') + 1
            col10_idx = headers.index('Col 10') + 1

            for r in range(2, last_row + 1):
                if ws.range((r, col7_idx)).value == 'Quantity':
                    current_val = ws.range((r, col10_idx)).value
                    if isinstance(current_val, (int, float)):
                        ws.range((r, col10_idx)).value = current_val * 1.20
            print("Demand increased by 20% successfully.")

        elif selected_decision_index == 1:
            # DECISION 1: Decrease Transport Costs by 15%
            print(f"Applying {decision_options[selected_decision_index]}...")
```

```

ws = wb.sheets['Paths']

last_col = ws.used_range.last_cell.column
last_row = ws.used_range.last_cell.row

headers_row = ws.range((1, 1), (1, last_col)).value
headers = headers_row if isinstance(headers_row, list) else [headers_row]

if 'Col 3' in headers and 'Col 4' in headers:
    col3_idx = headers.index('Col 3') + 1
    col4_idx = headers.index('Col 4') + 1

    for r in range(2, last_row + 1):
        val3 = ws.range((r, col3_idx)).value
        if val3 in ['Cost per unit', 'Cost']:
            current_val = ws.range((r, col4_idx)).value
            if isinstance(current_val, (int, float)):
                ws.range((r, col4_idx)).value = current_val * 0.85
            print("Transport costs decreased by 15% successfully.")

elif selected_decision_index == 2:
    # DECISION 2: Increase Safety Stock by 10%
    print(f"Applying {decision_options[selected_decision_index]}...")
    ws = wb.sheets['Inventory']

    last_col = ws.used_range.last_cell.column
    last_row = ws.used_range.last_cell.row

    headers_row = ws.range((1, 1), (1, last_col)).value
    headers = headers_row if isinstance(headers_row, list) else [headers_row]

    if 'Col 5' in headers and 'Col 6' in headers:
        col5_idx = headers.index('Col 5') + 1
        col6_idx = headers.index('Col 6') + 1

        for r in range(2, last_row + 1):
            if ws.range((r, col5_idx)).value == 'Safety stock':
                current_val = ws.range((r, col6_idx)).value
                if isinstance(current_val, (int, float)):
                    ws.range((r, col6_idx)).value = current_val * 1.10
            print("Safety stock increased by 10% successfully.")

# 3. Save changes explicitly using native Microsoft Excel
wb.save()
print(f"\nModified scenario safely saved to: {MODIFIED_EXCEL_PATH}")
print("Ready for ALX API Import.")

except Exception as e:
    print(f"An error occurred during native Excel modification: {e}")
finally:
    # 4. Always ensure the hidden Excel application is closed, even if it crashes
    try:
        wb.close()
    except: pass
    app.quit()

```

Reading original scenario from: C:\Users\Luis\Downloads\TFG\API\Global Network Examination.xlsx

Created a clean copy of the scenario.

Opening Microsoft Excel in the background to preserve strict formatting...

Applying DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)...

Safety stock increased by 10% successfully.

Modified scenario safely saved to: C:\Users\Luis\Downloads\TFG\API\Modified_Global Network Examination.xlsx

Ready for ALX API Import.

3. Modification by Excel

```
In [28]: # Create a unique name for the new scenario based on the AI decision
new_scenario_name = f"Modified_AI_Decision_{selected_decision_index}"
print(f"Importing '{MODIFIED_EXCEL_PATH}' as new scenario: '{new_scenario_name}'")

try:
    # We use import_excel to upload the file and create a new scenario.
    # Note: In most Python OpenAPI clients, passing the file path as a string is
    import_response = api_instance.import_excel(
        new_scenario_name=new_scenario_name,
        project_id=the_project.id,
        file=MODIFIED_EXCEL_PATH,
        need_to_import_experiments=True # We want to keep the experiments config
    )

    print("Import process initiated successfully.")
    pprint(import_response)

    # Since import_excel is asynchronous, we need to track the job_id until it f
    if import_response.job_id:
        print(f"\nTracking import job ID: {import_response.job_id}...")

        while True:
            # Get the current status of the import job
            import_status = api_instance.get_import_status(import_response.job_id)
            print(f"Current import status: {import_status.status}")

            # Check if the process has reached a final state
            if import_status.status in ['DONE', 'FAILED', 'CANCELED']:
                print(f"Import process finished with status: {import_status.status}")

                if import_status.status == 'DONE':
                    # We save the ID of the newly created scenario for the next
                    new_scenario_id = import_status.scenario_id
                    print(f"\nSUCCESS! New scenario created with ID: {new_scenario_id}")
                else:
                    # In case it fails, it usually provides an error message
                    print(f"Error details (if any): {getattr(import_status, 'error_message', None)}")
                    break

            # Wait for 2 seconds before checking again
            time.sleep(2)
        else:
            print("Warning: No job_id received from the import response.")
```

```
except Exception as e:
    print(f"Exception when calling OpenApiApi->import_excel: {e}")
```

Importing 'C:\Users\Luis\Downloads\TFG\API\Modified_Global Network Examination.xlsx' as new scenario: 'Modified_AI_Decision_2'...

Import process initiated successfully.

ApiImportResponse(error=None, job_id=120)

Tracking import job ID: 120...

Current import status: DONE

Import process finished with status: DONE

SUCCESS! New scenario created with ID: 275557

4. Run simulation on AI scenario

```
In [31]: # Run simulation on the new AI scenario
# The AI automatically configures a SIMULATION experiment for its newly created
# and runs it synchronously to generate the results.

print(f"--- PHASE 2 - BLOCK 4: RUNNING SIMULATION FOR NEW AI SCENARIO (ID: {new_

try:
    print("1. Fetching SIMULATION configuration for the imported scenario...")
    # Retrieve experiments explicitly for the AI scenario
    ai_run_configurations = api_instance.get_experiments(the_project.id, new_sce

    # Select the 'SIMULATION' type experiment
    ai_simulation_RC = next((r for r in ai_run_configurations if r.type == 'SIMU

    if ai_simulation_RC:
        print(f"Found SIMULATION experiment (ID: {ai_simulation_RC.id}). Launchi

        # 2. Run the simulation
        sim_result = api_instance.run_experiment_synchronously(ai_simulation_RC.
        # Safely extract the result ID
        the_ai_sim_result_id = getattr(sim_result, 'experiment_result_id', None)

        if the_ai_sim_result_id:
            print(f"\n[SYSTEM] Simulation finished! Captured Result ID: {the_ai_

            # 3. Fetch the full list of dashboard pages (KPIs) and store them fo
            print("\n3. Fetching dashboard pages layout...")
            ai_dashboard_pages = api_instance.get_experiment_dashboard_pages(the
            print(f"Found {len(ai_dashboard_pages)} dashboard pages ready for ex

        else:
            print("Failed to capture 'experiment_result_id' from the synchronous
            pprint(sim_result)
    else:
        print("No SIMULATION experiment found for the AI scenario.")

except Exception as e:
    print("Exception during Simulation Execution: %s\n" % e)
```

--- PHASE 2 - BLOCK 4: RUNNING SIMULATION FOR NEW AI SCENARIO (ID: 275557) ---

1. Fetching SIMULATION configuration for the imported scenario...

Found SIMULATION experiment (ID: 276244). Launching synchronously...

[SYSTEM] Simulation finished! Captured Result ID: 276773

3. Fetching dashboard pages layout...

Found 7 dashboard pages ready for export.

5. Export the new dashboard pages

```
In [32]: # Export results as an Excel matrix
# The AI Loops through every available dashboard page (Finance, Inventory, etc.)
# and saves each one as a separate Excel file to preserve visual charts.

print(f"--- PHASE 2 - BLOCK 5: EXPORTING AI SIMULATION RESULTS ---")

try:
    if 'ai_dashboard_pages' in locals() and len(ai_dashboard_pages) > 0:
        print(f"Exporting ALL dashboard pages from AI result ID {the_ai_sim_resu

        # Iterate over every page available in the dashboard
        for page in ai_dashboard_pages:
            # Clean the page name to make it a safe Windows filename (replace sp
            safe_page_name = re.sub(r'^a-zA-Z0-9_\-', '_', page.name)

            # Use the selected_decision_index to name the file accordingly
            # Create a unique filename for each page, identifying it's from the
            output_filename = f"AI_Decision{selected_decision_index}_Results_{sa
            output_path = os.path.join(os.getcwd(), output_filename)

            print(f"Downloading '{page.name}' (ID: {page.id}) -> {output_filenam

            # Export this specific page
            excel_export = api_instance.export_dashboard_page(the_ai_sim_result_

            # Save the file securely (handle both string paths and byte streams)
            if isinstance(excel_export, str) and os.path.exists(excel_export):
                shutil.move(excel_export, output_path)
            elif isinstance(excel_export, bytes):
                with open(output_path, "wb") as file:
                    file.write(excel_export)
            else:
                print(f"Unexpected data format received for page {page.name}")

            print("\n[TOTAL SUCCESS] All AI dashboard pages exported successfully.")
        else:
            print("No dashboard pages available to export.")

except Exception as e:
    print("Exception when exporting AI dashboard pages: %s\n" % e)
```

--- PHASE 2 - BLOCK 5: EXPORTING AI SIMULATION RESULTS ---

Exporting ALL dashboard pages from AI result ID 276773...

Downloading 'Available Inventory' (ID: 276775) -> AI_Decision2_Results_Available_Inventory.xlsx

Downloading 'Fulfillment' (ID: 276781) -> AI_Decision2_Results_Fulfillment.xlsx

Downloading 'Lead Time' (ID: 276788) -> AI_Decision2_Results_Lead_Time.xlsx

Downloading 'Production' (ID: 276792) -> AI_Decision2_Results_Production.xlsx

Downloading 'Profit and Loss Statement' (ID: 276802) -> AI_Decision2_Results_Profit_and_Loss_Statement.xlsx

Downloading 'Service Level' (ID: 276806) -> AI_Decision2_Results_Service_Level.xlsx

Downloading 'Tariffs' (ID: 276813) -> AI_Decision2_Results_Tariffs.xlsx

[TOTAL SUCCESS] All AI dashboard pages exported successfully.

=====

DEVELOPMENT TESTS - IGNORE THEM

◀ ————— ▶

```
In [ ]: #Avoid self-signed warnings
import os
import urllib3
```

For self-signed certificates it is necessary to download them and add to the certification path

```
In [ ]: import ssl
import socket

def download_self_signed_cert_no_validation(hostname, port, cert_file):
    context = ssl._create_unverified_context()
    conn = context.wrap_socket(socket.socket(socket.AF_INET), server_hostname=hostname)
    conn.connect((hostname, port))
    der_cert = conn.getpeercert(binary_form=True)
    pem_cert = ssl.DER_cert_to_PEM_cert(der_cert)

    with open(cert_file, 'w') as cert_file:
        cert_file.write(pem_cert)

    print(f"Certificate saved to {cert_file.name}")

# Example usage
download_self_signed_cert_no_validation(SERVER_IP, 443, MY_CERT_FILENAME)
```

```
In [ ]: # !/usr/bin/openssl s_client -showcerts -connect {SERVER_IP}:443 </dev/null 2>/d
```

```
In [ ]: MY_CERT_FILE=os.path.abspath(MY_CERT_FILENAME)
if os.path.isfile(MY_CERT_FILE):
    print(f"Cert file OK: {MY_CERT_FILE} ")
else:
    print(f"Cert file ERROR: {MY_CERT_FILE} ")
MY_CERT_FILE
```

```
In [ ]: os.environ['REQUESTS_CA_BUNDLE'] = MY_CERT_FILE

# Create a PoolManager with a custom CA bundle
http = urllib3.PoolManager(
    cert_reqs='CERT_NONE',      # Enforce cert verification
    ca_certs=MY_CERT_FILE       # Path to your self-signed cert
)

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
http.request("GET", TEST_URL)
```

```
In [ ]: from urllib3 import HTTPSConnectionPool

pool = HTTPSConnectionPool(
    SERVER_IP,
    port=443,
    cert_reqs='CERT_REQUIRED',
    ca_certs=MY_CERT_FILE
)
```

```
In [ ]: import requests

# Any request you make now will use your certificate
response = requests.get(TEST_URL, verify=False)
print(response.text)
```